Lab Sheet 5: SQL Joins

Solved queries with sample outputs — MySQL 8.x

Note: Sample outputs below are based on the small example dataset in the appendix. MySQL does not directly support FULL OUTER JOIN, so question 12 uses a UNION workaround. For the course-enrolment questions, the sample uses an `Enrollment` table.

Tables and relationships used

Department(DepartmentID, DepartmentName); Student(StudentID, StudentName, DepartmentID, CourseID); Course(CourseID, CourseName); Employee(EmployeeID, EmployeeName, Salary, DepartmentID, ManagerID); Enrollment(StudentID, CourseID).

1. Demonstrate an INNER JOIN between Student and Department.

SQL query:

```
SELECT s.StudentID, s.StudentName, d.DepartmentName
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
StudentID | StudentName | DepartmentName
1 | Aarav | Computer Science
2 | Diya | AI & Data Science
3 | Kabir | Computer Science
4 | Meera | Electronics
```

2. Display student names along with their department names using INNER JOIN.

SQL query:

```
SELECT s.StudentName, d.DepartmentName
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
StudentName | DepartmentName
Aarav | Computer Science
Diya | AI & Data Science
Kabir | Computer Science
Meera | Electronics
```

3. Display employee names along with their department names.

SQL query:

```
SELECT e.EmployeeName, d.DepartmentName
FROM Employee e
INNER JOIN Department d ON e.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
EmployeeName | DepartmentName
Rohan | Computer Science
Simran | AI & Data Science
Ankit | Computer Science
Priya | Electronics
```

4. Display students along with their course details.

SQL query:

```
SELECT s.StudentName, c.CourseID, c.CourseName
FROM Student s
INNER JOIN Course c ON s.CourseID = c.CourseID;
```

Sample output / explanation:

```
StudentName | CourseID | CourseName
Aarav | 101 | DBMS
Diya | 102 | Python
Kabir | 101 | DBMS
Meera | 103 | Java
```

5. Find students belonging to a particular department using INNER JOIN.

SQL query:

```
SELECT s.StudentName, d.DepartmentName
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'Computer Science';
```

Sample output / explanation:

```
StudentName | DepartmentName
Aarav | Computer Science
Kabir | Computer Science
```

6. Demonstrate a LEFT OUTER JOIN between Student and Department.

SQL query:

```
SELECT s.StudentName, d.DepartmentName
FROM Student s
LEFT OUTER JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
StudentName | DepartmentName
Aarav | Computer Science
Diya | AI & Data Science
Kabir | Computer Science
Meera | Electronics
Ishita | NULL
```

7. Display all students, including those who do not have a matching department.

SQL query:

```
SELECT s.StudentID, s.StudentName, d.DepartmentName
FROM Student s
LEFT JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
StudentID | StudentName | DepartmentName
1 | Aarav | Computer Science
2 | Diya | AI & Data Science
3 | Kabir | Computer Science
4 | Meera | Electronics
5 | Ishita | NULL
```

8. Demonstrate a RIGHT OUTER JOIN between Student and Department.

SQL query:

```
SELECT s.StudentName, d.DepartmentName
FROM Student s
RIGHT OUTER JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
StudentName | DepartmentName
Aarav | Computer Science
Kabir | Computer Science
Diya | AI & Data Science
Meera | Electronics
NULL | Mechanical
```

9. Display all departments, including departments having no students.

SQL query:

```
SELECT d.DepartmentName, s.StudentName
FROM Department d
LEFT JOIN Student s ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
DepartmentName | StudentName
Computer Science | Aarav
Computer Science | Kabir
AI & Data Science | Diya
Electronics | Meera
Mechanical | NULL
```

10. Find students who are not assigned to any department using LEFT JOIN.

SQL query:

```
SELECT s.StudentName
FROM Student s
LEFT JOIN Department d ON s.DepartmentID = d.DepartmentID
WHERE d.DepartmentID IS NULL;
```

Sample output / explanation:

```
StudentName
Ishita
```

11. Find departments having no students using RIGHT/LEFT JOIN.

SQL query:

```
SELECT d.DepartmentName
FROM Department d
LEFT JOIN Student s ON s.DepartmentID = d.DepartmentID
WHERE s.StudentID IS NULL;
```

Sample output / explanation:

```
DepartmentName
Mechanical
```

12. Demonstrate a FULL OUTER JOIN concept using a MySQL workaround.

SQL query:

```
SELECT s.StudentName, d.DepartmentName
FROM Student s LEFT JOIN Department d ON s.DepartmentID = d.DepartmentID
UNION
SELECT s.StudentName, d.DepartmentName
FROM Student s RIGHT JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
StudentName | DepartmentName
Aarav | Computer Science
Kabir | Computer Science
Diya | AI & Data Science
Meera | Electronics
Ishita | NULL
NULL | Mechanical
```

13. Demonstrate a CROSS JOIN between Student and Course.

SQL query:

```
SELECT s.StudentName, c.CourseName
FROM Student s CROSS JOIN Course c;
```

Sample output / explanation:

```
Creates every student-course pair. With 5 students and 3 courses, 15 rows are returned.
Examples:
Aarav | DBMS
Aarav | Python
Aarav | Java
Diya | DBMS
... (remaining combinations)
```

14. Calculate the number of possible Student-Course combinations using CROSS JOIN.

SQL query:

```
SELECT COUNT(*) AS TotalCombinations
FROM Student CROSS JOIN Course;
```

Sample output / explanation:

```
TotalCombinations
15
```

15. Demonstrate a NATURAL JOIN between two suitable tables.

SQL query:

```
SELECT StudentName, DepartmentName
FROM Student NATURAL JOIN Department;
```

Sample output / explanation:

```
StudentName | DepartmentName
Aarav | Computer Science
Diya | AI & Data Science
Kabir | Computer Science
Meera | Electronics
Note: NATURAL JOIN matches same-named columns automatically.
```

16. Compare the result of NATURAL JOIN and INNER JOIN.

SQL query:

```
SELECT s.StudentName, d.DepartmentName
FROM Student s INNER JOIN Department d
ON s.DepartmentID = d.DepartmentID;

-- Compare with:
SELECT StudentName, DepartmentName
FROM Student NATURAL JOIN Department;
```

Sample output / explanation:

```
Both return the four students with matching departments in this dataset. INNER JOIN states the condition explicitly;
NATURAL JOIN automatically matches columns with the same names.
```

17. Perform a SELF JOIN on Employee to show employee-manager relationships.

SQL query:

```
SELECT e.EmployeeName AS Employee, m.EmployeeName AS Manager
FROM Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmployeeID;
```

Sample output / explanation:

```
Employee | Manager
Rohan | NULL
Simran | Rohan
Ankit | Rohan
Priya | Simran
```

18. Display employees along with their managers using SELF JOIN.

SQL query:

```
SELECT e.EmployeeName, COALESCE(m.EmployeeName, 'No manager') AS ManagerName
FROM Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmployeeID;
```

Sample output / explanation:

```
EmployeeName | ManagerName
Rohan | No manager
Simran | Rohan
Ankit | Rohan
Priya | Simran
```

19. Join three tables—Student, Department and Course.

SQL query:

```
SELECT s.StudentName, d.DepartmentName, c.CourseName
FROM Student s
JOIN Department d ON s.DepartmentID = d.DepartmentID
JOIN Course c ON s.CourseID = c.CourseID;
```

Sample output / explanation:

```
StudentName | DepartmentName | CourseName
Aarav | Computer Science | DBMS
Diya | AI & Data Science | Python
Kabir | Computer Science | DBMS
Meera | Electronics | Java
```

20. Display student name, course name and department name using multiple joins.

SQL query:

```
SELECT s.StudentName, c.CourseName, d.DepartmentName
FROM Student s
JOIN Course c ON s.CourseID = c.CourseID
JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Sample output / explanation:

```
StudentName | CourseName | DepartmentName
Aarav | DBMS | Computer Science
Diya | Python | AI & Data Science
Kabir | DBMS | Computer Science
Meera | Java | Electronics
```

21. Find employees whose salary is greater than the average salary of their department.

SQL query:

```
SELECT e.EmployeeName, e.Salary, e.DepartmentID
FROM Employee e
JOIN (SELECT DepartmentID, AVG(Salary) AS AvgSalary
FROM Employee GROUP BY DepartmentID) a
ON e.DepartmentID = a.DepartmentID
WHERE e.Salary > a.AvgSalary;
```

Sample output / explanation:

```
EmployeeName | Salary | DepartmentID
Ankit | 65000.00 | 1
Priya | 58000.00 | 3
```

22. Find the department having the maximum number of students using joins and aggregation.

SQL query:

```
SELECT d.DepartmentName, COUNT(s.StudentID) AS StudentCount
FROM Department d LEFT JOIN Student s ON s.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName
ORDER BY StudentCount DESC LIMIT 1;
```

Sample output / explanation:

```
DepartmentName | StudentCount
Computer Science | 2
```

23. Display departments and the total number of students in each department.

SQL query:

```
SELECT d.DepartmentName, COUNT(s.StudentID) AS StudentCount
FROM Department d LEFT JOIN Student s ON s.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName
ORDER BY d.DepartmentID;
```

Sample output / explanation:

```
DepartmentName | StudentCount
Computer Science | 2
AI & Data Science | 1
Electronics | 1
Mechanical | 0
```

24. Display courses along with the number of students enrolled in each course.

SQL query:

```
SELECT c.CourseName, COUNT(e.StudentID) AS EnrolledStudents
FROM Course c LEFT JOIN Enrollment e ON e.CourseID = c.CourseID
GROUP BY c.CourseID, c.CourseName
ORDER BY c.CourseID;
```

Sample output / explanation:

```
CourseName | EnrolledStudents
DBMS | 2
Python | 2
Java | 1
```

25. Create a complete query using joins, GROUP BY and aggregate functions for a student-department-course report.

SQL query:

```
SELECT d.DepartmentName, c.CourseName,
COUNT(s.StudentID) AS StudentCount
FROM Department d
LEFT JOIN Student s ON s.DepartmentID = d.DepartmentID
LEFT JOIN Course c ON s.CourseID = c.CourseID
GROUP BY d.DepartmentName, c.CourseName
ORDER BY d.DepartmentName, c.CourseName;
```

Sample output / explanation:

```
DepartmentName | CourseName | StudentCount
AI & Data Science | Python | 1
Computer Science | DBMS | 2
Electronics | Java | 1
Mechanical | NULL | 0
```

Appendix: Create and populate the sample tables

Run this setup SQL once in MySQL Workbench before running the queries. If your own table or column names differ, adjust the queries accordingly.

```
CREATE TABLE Department (
DepartmentID INT PRIMARY KEY,
DepartmentName VARCHAR(60) NOT NULL
);
INSERT INTO Department VALUES
(1, 'Computer Science'), (2, 'AI & Data Science'),
(3, 'Electronics'), (4, 'Mechanical');

CREATE TABLE Course (
CourseID INT PRIMARY KEY,
CourseName VARCHAR(50) NOT NULL
);
INSERT INTO Course VALUES
(101, 'DBMS'), (102, 'Python'), (103, 'Java');

CREATE TABLE Student (
StudentID INT PRIMARY KEY,
StudentName VARCHAR(50) NOT NULL,
DepartmentID INT NULL,
CourseID INT NULL
);
INSERT INTO Student VALUES
(1, 'Aarav', 1, 101), (2, 'Diya', 2, 102),
(3, 'Kabir', 1, 101), (4, 'Meera', 3, 103),
(5, 'Ishita', NULL, NULL);

CREATE TABLE Employee (
EmployeeID INT PRIMARY KEY,
EmployeeName VARCHAR(50) NOT NULL,
Salary DECIMAL(10,2),
DepartmentID INT,
ManagerID INT NULL
);
INSERT INTO Employee VALUES
(201, 'Rohan', 70000, 1, NULL),
(202, 'Simran', 60000, 2, 201),
(203, 'Ankit', 65000, 1, 201),
(204, 'Priya', 58000, 3, 202);

CREATE TABLE Enrollment (
StudentID INT,
CourseID INT,
PRIMARY KEY (StudentID, CourseID)
);
INSERT INTO Enrollment VALUES
(1,101), (2,102), (3,101), (4,103), (5,102);
```